ATMIYA UNIVERSITY
RAJKOT, GUJARAT (INDIA)

RIT

ADSC

ATMIYA DEVELOPER STUDENTS CLUB PRESENTS

CODE CARNIVAL 3.0

Idea Submission · AegisRAG: answers only from the files you are allowed to read

TEAM NAME	TEAM ID	TEAM LEADER
The Boor Bots	SJX3	Navadeep Gandloju

PROBLEM STATEMENT TITLE	THEME / TRACK
PS-01 · Secure Multi-Modal RAG System with Access Control	AI Security · Enterprise RAG

01 · THE TARGET

PROBLEM STATEMENT & UNDERSTANDING

PROBLEM STATEMENT

- Companies want one AI assistant over all files, but each file has its own readers
- Typical RAG searches everything, then filters, so restricted text can reach the model
- Access rules must hold across PDFs, scans, photos and spreadsheets

OUR UNDERSTANDING

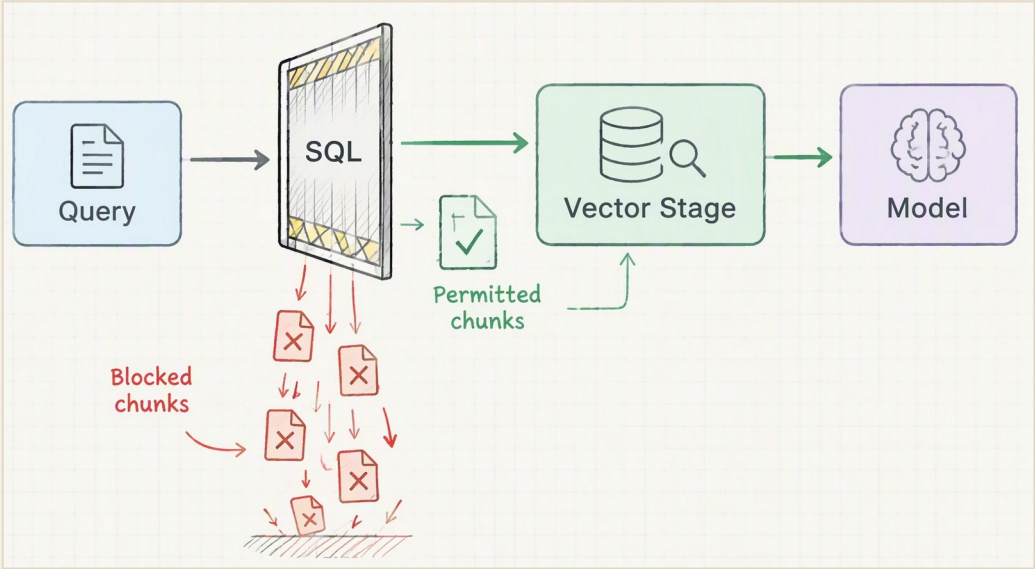
- Root cause: permissions are checked after retrieval or written into the prompt
- Prompt rules can be bypassed by injected text such as “ignore previous instructions”
- LLMs guess totals over tables and cite sources that do not say it
- Revoking access must also hide answers given earlier

02 · THE PLAN

PROPOSED SOLUTION

OUR IDEA

- AegisRAG compiles each user's permissions into the retrieval SQL query
- Restricted chunks are never loaded, ranked or sent to the model
- Totals over spreadsheets are computed in code, only on rows the user may read
- Every answer cites its source; unsupported claims are removed



Deny by default: blocked chunks are filtered inside the database

03 · THE ARSENAL

KEY FEATURES / FUNCTIONALITY

01 **Access rule inside SQL**

Tenant, grant, clearance and row rules filter every search, count and citation

02 **Multimodal ingestion**

PDF, scans, photos, DOCX, XLSX, CSV and JSON; OCR via Gemini Vision or RapidOCR

03 **Exact answers over records**

Sums, averages and group-bys run in code on only the rows each user may see

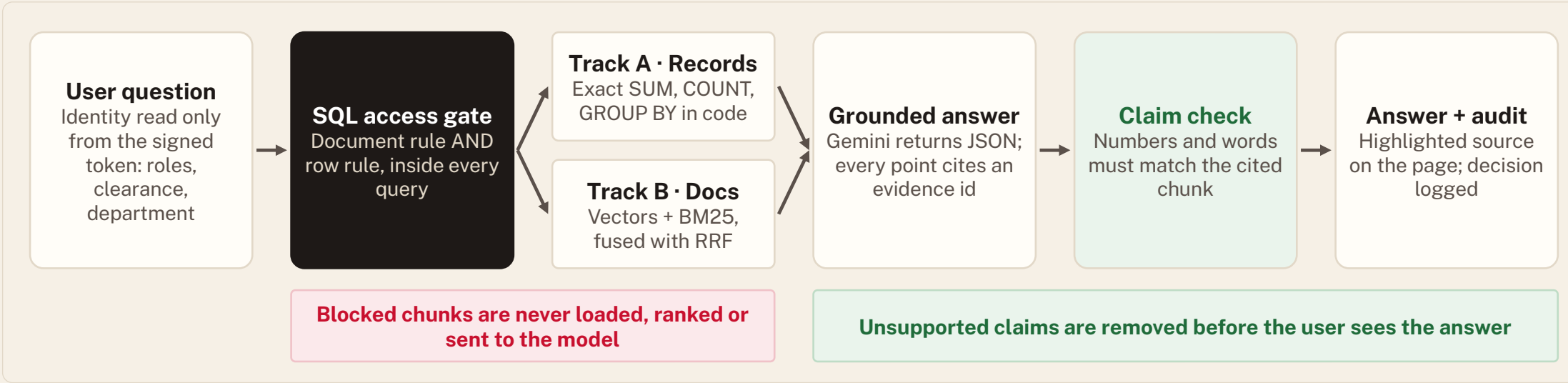
04 **Verified citations and audit**

Each claim is checked against its cited chunk; revoked access withdraws old answers

04 · THE EXECUTION

HOW THE SOLUTION WILL WORK

STEP 1 Sign in: signed JWT → **STEP 2** SQL permission filter → **STEP 3** Search or compute → **STEP 4** Cited, verified answer



05 · THE GEAR

TECHNOLOGY / TECH STACK

Layer	Technology	Why we chose it
Frontend	React 18, TypeScript, Vite, Tailwind	Typed UI, fast builds
Backend	Python, FastAPI, SQLAlchemy	Async API, signed identity
Database	PostgreSQL + pgvector, SQLite	Vectors and access rule in one query
AI / APIs	Gemini API, RapidOCR	Grounded JSON; offline OCR fallback
Hosting	Render, Docker Compose	One-file deploy via render.yaml
Tools	GitHub, pytest, PyMuPDF, openpyxl	97 automated tests guard the rules

06 · INSIDE THE MINT

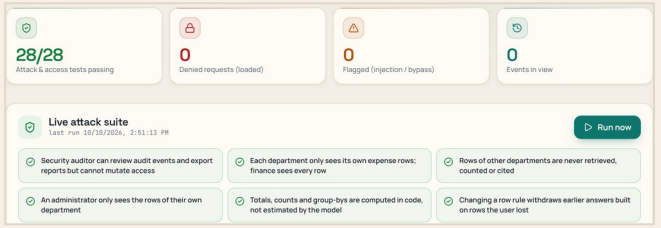
CURRENT PROGRESS / WORK DONE

COMPLETED

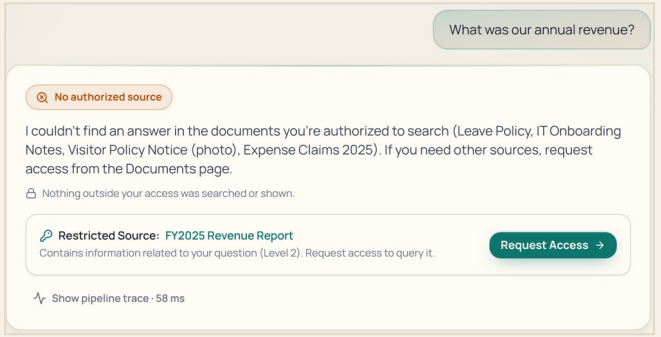
- Access rule in SQL, row rules, multi-tenant
- Ingestion: PDF, scans, photos, DOCX, sheets
- Hybrid search and exact analytics
- 97 / 97 backend tests passing

IN PROGRESS

- Gemini tuning within free-tier rate limits
- Cleaner tables in answers
- Catching rephrased injection (pattern-based today)



Live attack suite: 28 / 28 passing



Employee asks for revenue: refused, no leak

07 · THE GETAWAY

FUTURE PLAN / NEXT STEPS

PHASE 1

DURING THE HACKATHON

- Deploy live with Gemini on Render
- Fix answer table layout
- Run the attack suite live in the demo

PHASE 2

RIGHT AFTER

- Okta / Entra ID sign-in with SCIM role sync
- Reranker after RRF for sharper results
- Malware scanning of uploads

PHASE 3

LONG-TERM VISION

- Audio and meeting transcripts with speaker-level access
- Pilot with one college or company department
- Per-seat SaaS plus a self-hosted option

08 · THE VAULT

GITHUB REPOSITORY LINK

https://github.com/Navadeep44/aegisrag-2

PUBLIC

Repository visibility is set to public.

README

Covers setup, features, demo accounts and tests.

COMMITS

Regular commits from both team members.

DEMO (OPTIONAL)

Run locally in two commands (see README); Render config included.